

Homework — Topic 07

Build a Simple Safe Agent

Autonomous Agents & Tool Integration

1. Homework Objective

Build a small agentic application that can receive a user request, choose and call a tool, observe the tool result, and continue until it can produce a final answer. The required version should be simple. The focus is understanding the agent loop and basic application-level permission control, not building a large production system.

Core idea: The model proposes an action. The application decides whether that action is allowed to execute.

2. Minimum Requirements — Required

A. Simple Agent Loop

Your agent must demonstrate the following flow:

User Request → Agent / LLM → Tool Call → Tool Execution → Result → Agent Decides Again

1. Receive a user request.
2. Decide whether a tool is needed.
3. Select one of the available tools.
4. Call the tool with structured arguments.
5. Receive and observe the tool result.
6. Decide what to do next.
7. Return a final answer when the goal is satisfied or the run must stop.

Use at least 2 tools. The tools may be simple local Python functions or database-backed functions.

B. Tool Implementation + Tool Schema

Each tool should have both an implementation and a defined input schema. The schema describes how the model may request the capability; the implementation describes what the application actually does.

- Tool implementation: the real Python function or business logic.
- Tool schema: the expected inputs, types, required fields, and any allowed values or constraints.
- You may use Pydantic, JSON Schema, or the schema mechanism provided by your framework.

C. Basic Permission Control

Your application must include at least one permission rule. Permission must be checked in application code, not only described in the prompt.

Action	Customer	Admin
search_products		
check_stock		
delete_product		

The example above is only a model. You may define your own roles and permissions.

D. Basic Safety

Your required project must include at least the following safety controls:

- Basic input validation (for example, positive IDs or valid quantities).
- Basic error handling with a controlled error result or message.
- A maximum iteration or tool-call limit to prevent an endless agent loop.

3. Choose a Small Project

Choose a small problem where an agent can call a few tools. You may use one of the examples below or create your own.

Example project	Possible tools
Shopping Agent	search_product(), check_stock(), buy_product()
Student Assistant	search_course(), check_schedule(), register_course()
Library Agent	search_book(), check_availability(), borrow_book()
Restaurant Agent	search_restaurant(), check_menu(), make_reservation()

Keep the scope small. A working 2–4 tool agent is better than a large unfinished system.

4. Recommended Architecture

A simple project structure is enough. You do not need a production-grade architecture.

```
my-agent/
├── README.md
├── main.py
├── agent.py
├── tools.py
├── schemas.py
└── harness.py  # optional, but recommended
```

Suggested responsibility of each file:

- main.py — starts the application and accepts the user request.

- `agent.py` — model, agent loop, tool registration/exposure, and routing.
- `tools.py` — actual tool implementations/business logic.
- `schemas.py` — explicit tool input schemas.
- `harness.py` — permission, validation, limits, and safety checks (optional but recommended).

5. Example of the Required Behavior

Example: a shopping agent receives “Find a laptop that is currently in stock.”

```
User
↓
Agent / LLM
↓
search_products("laptop")
↓
Tool result: candidate products
↓
Agent observes result
↓
check_stock(product_id=2)
↓
Tool result: stock = 5
↓
Agent
↓
Final answer
```

The exact domain and tool names may be different in your project. The important part is that the agent can use the result of one action to determine whether another action is needed.

6. Optional Bonus / Advanced Extensions

These features are NOT required. They are available for students who want to extend the basic project and demonstrate deeper understanding.

- **Human-in-the-Loop (HITL):** Require a human to approve a sensitive action before execution, such as a purchase, deletion, or external message.
- **Risk Classification:** Classify tools as Green / Yellow / Red and use different execution policies.
- **Allowlist:** Reject any tool that is not explicitly approved by the application.
- **Stronger Failure Boundaries:** Return structured errors such as `OUT_OF_STOCK` instead of exposing raw exceptions, then let the agent observe and recover.
- **Retry / Timeout Controls:** Add bounded retries, tool timeouts, rate limits, or a maximum tool-call count.
- **MCP Server:** Expose one or more tools through an MCP server and explain the Host → Client → Server → Tool path.
- **Advanced Agent Pattern:** Extend the basic loop with ReAct, Router, Planner–Executor, Reflection / Evaluator, or Multi-Agent delegation.

For advanced extensions, be ready to explain why you added the feature and what problem it solves.

7. README Requirements

Your README.md should contain the following sections:

1. Project Overview — What does your agent do?
2. Available Tools — List the tools and briefly explain each one.
3. Agent Loop — Show the decision → action → observation flow.
4. Permission Rule — Explain who can perform which action.
5. Safety — Explain your validation, error handling, and loop limit.
6. Example Run — Show at least one real input and the resulting tool calls/output.

8. Submission

Submit the following:

- Source code. (github repo)
- README.md.
- The screenshot or terminal output showing the agent running with your testcase example.
- Any additional files required to run the project.

Your README should include clear run instructions so that the instructor can start the project.

9. Submission Checklist

- At least 2 tools are available.
- The model can choose and call a tool.
- The tool result is returned to the agent.
- The agent can continue the loop after observing a result.
- Tool inputs have a defined schema.
- At least one permission rule is enforced in application code.
- Basic validation is implemented.
- Basic error handling is implemented.
- A maximum iteration/tool-call limit is implemented.
- README and run instructions are included.

10. What Matters Most

Required work: A small, working agent loop with basic permission and safety controls.

Advanced work: HITL, risk tiers, stronger failure handling, MCP, or advanced agent patterns are optional and may earn bonus recognition.

Do not make the project unnecessarily large. The objective is to show that you understand the architecture and control flow of a tool-using agent.

Course Alignment

This homework is aligned with Topic 07 — Autonomous Agents & Tool Integration, especially the agent loop, tool calling, structured function invocation, bounded actions and safety, harness/control concepts, MCP, and human-in-the-loop design. The optional features extend beyond the minimum requirements so stronger students can demonstrate additional understanding.